Homework 9

¶

¶ Regression with Nonlinear Effects ¶

¶

¶ < JACOB FISHER > ¶

Data and Description

See the attached data description.

0. Replace the text “< PUT YOUR NAME HERE >” (above next to “author:”) with your full name.

1. Read in the data set, and call the data frame `snow`. Make sure the core variable is a factor (if they are not, you’ll need to make them factors). Print a summary of the data and make sure the data makes sense.

```
snow <- read.csv("snow_core.csv") %>%
  mutate(Core = as.factor(Core))
summary(snow)
```

```
##       Core          Depth          Density
##  3      :1128   Min.   : 0.00   Min.   :0.2290
##  13     :1008   1st Qu.: 4.32   1st Qu.:0.4752
##  11     : 899   Median : 8.66   Median :0.5345
##  7      : 895   Mean   : 8.78   Mean   :0.5163
##  5      : 893   3rd Qu.:13.00   3rd Qu.:0.5703
##  8      : 867   Max.   :22.54   Max.   :0.6705
##  (Other):6447
```

2. What is your outcome? What is/are your explanatory variable(s)?

Density is the outcome variable with depth and core number as our explanatory variables.

3. Fit a linear model using depth as a linear effect. Also, include an effect for core BUT NO INTERACTION.

```
lm_snow <- lm(Density ~ Depth+Core ,data = snow)
lm_snow
```

```
##
## Call:
## lm(formula = Density ~ Depth + Core, data = snow)
##
## Coefficients:
## (Intercept)        Depth        Core2        Core3        Core4        Core5
##   0.4163200    0.0120798    0.0006071   -0.0189423   -0.0056777   -0.0116790
##       Core6        Core7        Core8        Core9       Core10       Core11
##  -0.0171403   -0.0030338   -0.0041327    0.0031216    0.0014737   -0.0016919
##      Core12       Core13       Core14
##  -0.0026773   -0.0138113   -0.0046774
```

4. For this model, calculate the AIC.

```
AIC(lm_snow)
```

```
## [1] -50348.49
```

5. Fit a linear model using depth as a linear effect and an interaction with core. That is, a model with a line for each core. Use the predict function to save the model fit as `pred.linear` in the `snow` data frame. Save the residuals as `resid.linear` in the `snow` data frames. You will use these in problems 7 and 8.

```
lm_cores <- lm(Density ~ Depth+Depth*Core ,data = snow)
lm_cores
```

```
##
## Call:
## lm(formula = Density ~ Depth + Depth * Core, data = snow)
##
## Coefficients:
##  (Intercept)         Depth         Core2         Core3         Core4
##    4.206e-01     1.152e-02    -9.106e-04     1.852e-02    -1.753e-02
##        Core5         Core6         Core7         Core8         Core9
##   -2.170e-02    -5.722e-02    -1.472e-02    -3.724e-03     6.825e-03
##       Core10        Core11        Core12        Core13        Core14
##   -1.355e-03     5.209e-04    -1.667e-02    -3.527e-02    -3.066e-02
##  Depth:Core2   Depth:Core3   Depth:Core4   Depth:Core5   Depth:Core6
##    1.982e-04    -3.153e-03     1.470e-03     1.194e-03     4.936e-03
##  Depth:Core7   Depth:Core8   Depth:Core9  Depth:Core10  Depth:Core11
##    1.378e-03     7.918e-06    -3.779e-04     3.804e-04    -1.735e-04
## Depth:Core12  Depth:Core13  Depth:Core14
##    1.707e-03     2.256e-03     3.767e-03
```

```
snow$resid.linear<-resid((lm_cores))
```

6. For this model, calculate the AIC.

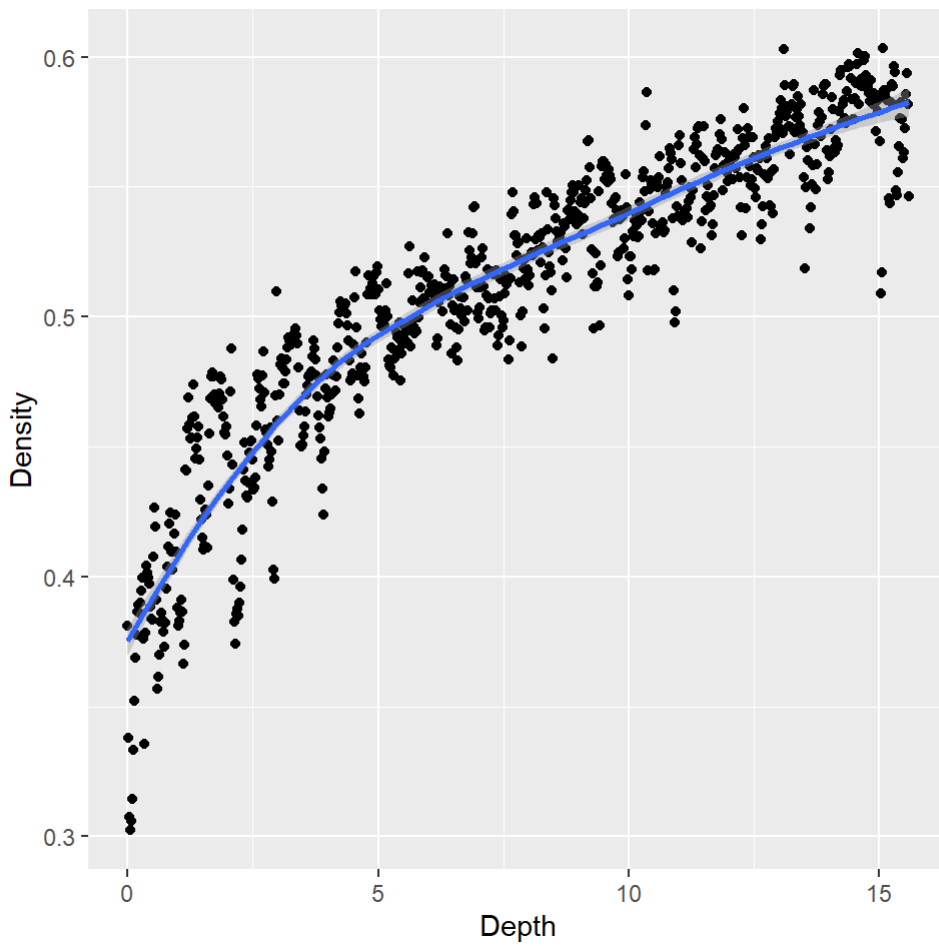
```
AIC(lm_cores)
```

```
## [1] -51851.09
```

7. For the first site (Core = 1), plot the data and your model fit together.

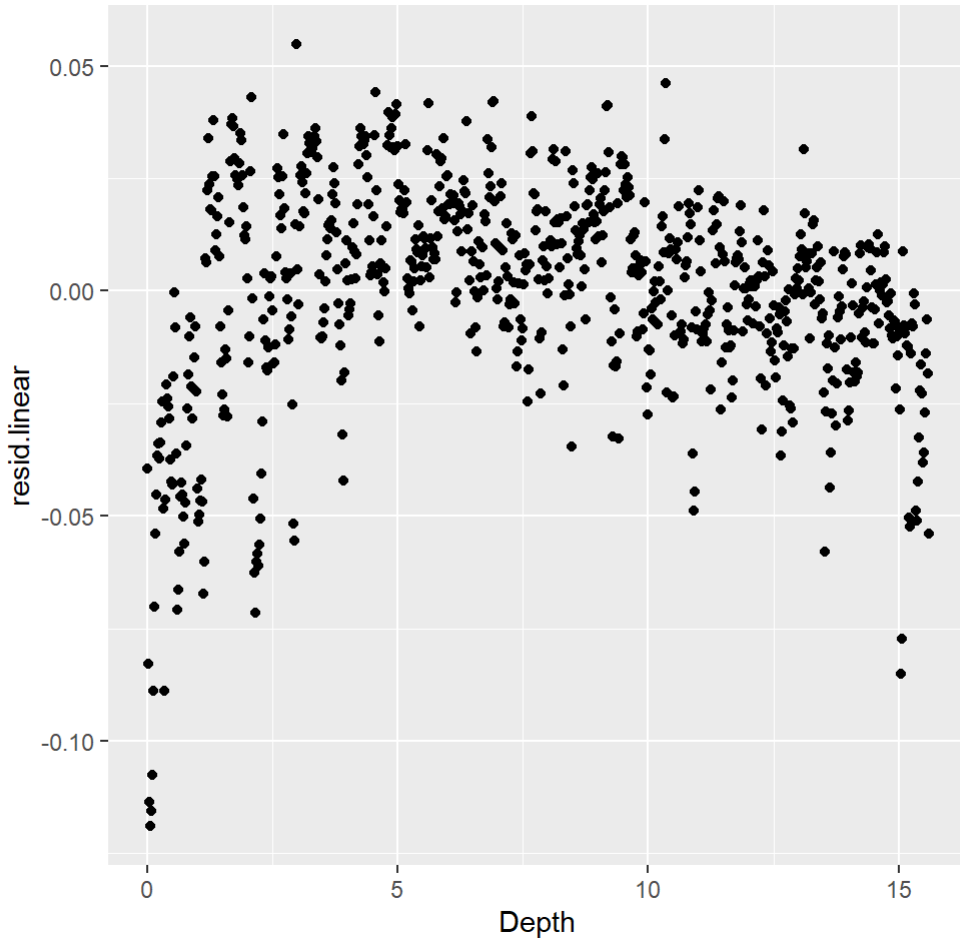
```
ggplot(subset(snow, Core == 1), aes(x = Depth, y = Density)) +
  geom_point() +
  geom_smooth() +
  theme(aspect.ratio = 1)
```

```
## `geom_smooth()` using method = 'loess' and formula 'y ~ x'
```



8. For the first site (Core = 1), plot the residuals against depth. Comment on the pattern.

```
# Create a scatter plot of residuals versus depth
ggplot(subset(snow, Core == 1), aes(x = Depth, y = resid.linear)) +
  geom_point() +
  theme(aspect.ratio = 1)
```



The residuals differ from zero more at the extremes and are more uniform towards the middle, though there is a seeming quadratic trend to the residuals.

9. Fit a model using a cubic polynomial for depth. Also, include an effect for core BUT NO INTERACTION.

```
lm_poly_cub = lm(Density ~ Depth + poly(Core,3),data = snow)
lm_poly_cub
```

```
##
## Call:
## lm(formula = Density ~ Depth + poly(Core, 3), data = snow)
##
## Coefficients:
##    (Intercept)           Depth  poly(Core, 3)1  poly(Core, 3)2  poly(Core, 3)3
##        0.41163         0.01193         0.10757        -0.01507        -0.48435
```

10. For this model, calculate the AIC.

```
AIC(lm_poly_cub)
```

```
## [1] -49999.52
```

11. Fit a model using a cubic polynomial for depth and an interaction with core. That is, a model with a unique cubic polynomial for each core. Something like `lm(Density ~ Core * poly(Depth,3))`.

```
lm_poly_cubcore<-lm(Density ~ Core * poly(Depth,3), data=snow)
lm_poly_cubcore
```

```
##
## Call:
## lm(formula = Density ~ Core * poly(Depth, 3), data = snow)
##
## Coefficients:
##            (Intercept)                   Core2                   Core3
##               0.522615               -0.002372               -0.006114
##                   Core4                   Core5                   Core6
##              -0.009084               -0.012279               -0.018886
##                   Core7                   Core8                   Core9
##              -0.003768               -0.003843                0.003308
##                  Core10                  Core11                  Core12
##               0.001585               -0.001048               -0.005146
##                  Core13                  Core14         poly(Depth, 3)1
##              -0.012816               -0.027209                6.856556
##        poly(Depth, 3)2         poly(Depth, 3)3   Core2:poly(Depth, 3)1
##              -0.363994                1.684281               -0.574869
##  Core3:poly(Depth, 3)1   Core4:poly(Depth, 3)1   Core5:poly(Depth, 3)1
##              -1.083914               -0.197791                0.319827
##  Core6:poly(Depth, 3)1   Core7:poly(Depth, 3)1   Core8:poly(Depth, 3)1
##               1.689941                0.397279               -0.109346
##  Core9:poly(Depth, 3)1  Core10:poly(Depth, 3)1  Core11:poly(Depth, 3)1
##              -0.330152                0.055844               -0.222309
## Core12:poly(Depth, 3)1  Core13:poly(Depth, 3)1  Core14:poly(Depth, 3)1
##               0.127955                1.770160               -3.307506
##  Core2:poly(Depth, 3)2   Core3:poly(Depth, 3)2   Core4:poly(Depth, 3)2
##              -1.051680               -1.399266               -1.821833
##  Core5:poly(Depth, 3)2   Core6:poly(Depth, 3)2   Core7:poly(Depth, 3)2
##              -1.370591               -1.946593               -1.430967
##  Core8:poly(Depth, 3)2   Core9:poly(Depth, 3)2  Core10:poly(Depth, 3)2
##              -1.005119               -1.061244               -1.344089
## Core11:poly(Depth, 3)2  Core12:poly(Depth, 3)2  Core13:poly(Depth, 3)2
##              -1.405378               -1.892769               -2.053382
## Core14:poly(Depth, 3)2   Core2:poly(Depth, 3)3   Core3:poly(Depth, 3)3
##              -5.294183               -0.772865               -1.148765
##  Core4:poly(Depth, 3)3   Core5:poly(Depth, 3)3   Core6:poly(Depth, 3)3
##              -1.781186               -1.534820               -1.813082
##  Core7:poly(Depth, 3)3   Core8:poly(Depth, 3)3   Core9:poly(Depth, 3)3
##              -1.709631               -0.106994                0.019275
## Core10:poly(Depth, 3)3  Core11:poly(Depth, 3)3  Core12:poly(Depth, 3)3
##               0.101054               -0.474902               -1.511324
## Core13:poly(Depth, 3)3  Core14:poly(Depth, 3)3
##              -0.845584               -3.893740
```

12. For this model, calculate the AIC.

```
AIC(lm_poly_cubcore)
```

```
## [1] -57519.7
```

13. Fit a model using a cubic spline for depth. Place a knot at 10 meters depth. Also, include an effect for core BUT NO INTERACTION.

```
lm_spline_cub = lm(Density ~ Core +
                      bs(Depth,knots = 10, degree = 3),
                    data = snow)
lm_spline_cub
```

```
##
## Call:
## lm(formula = Density ~ Core + bs(Depth, knots = 10, degree = 3),
##     data = snow)
##
## Coefficients:
##                        (Intercept)                              Core2
##                          3.641e-01                          7.668e-04
##                              Core3                              Core4
##                         -5.687e-03                         -4.753e-03
##                              Core5                              Core6
##                         -8.758e-03                         -1.636e-02
##                              Core7                              Core8
##                         -4.742e-05                         -2.017e-03
##                              Core9                             Core10
##                          5.091e-03                          3.386e-03
##                             Core11                             Core12
##                          1.426e-03                         -1.318e-03
##                             Core13                             Core14
##                         -6.484e-03                         -6.011e-03
## bs(Depth, knots = 10, degree = 3)1  bs(Depth, knots = 10, degree = 3)2
##                          1.102e-01                          2.100e-01
## bs(Depth, knots = 10, degree = 3)3  bs(Depth, knots = 10, degree = 3)4
##                          2.415e-01                          2.498e-01
```

14. For this model, calculate the AIC.

```
AIC(lm_spline_cub)
```

```
## [1] -55625.93
```

15. Fit a model using depth as cubic spline for depth and an interaction with core. Again, place a knot at 10 meters depth. That is, a model with a unique cubic spline for each. Use the predict function to save the model fit as `pred.spline` in the `snow` data frame. Save the residuals as `resid.spline` in the `snow` data frames.

```
lm_spline_cubint = lm(Density ~ Core + Core*Depth +
                      bs(Depth,knots = 10, degree = 3),
                    data = snow)
snow$pred.spline<-predict(lm_spline_cubint)
snow$resid.spline <- resid(lm_spline_cubint)
```

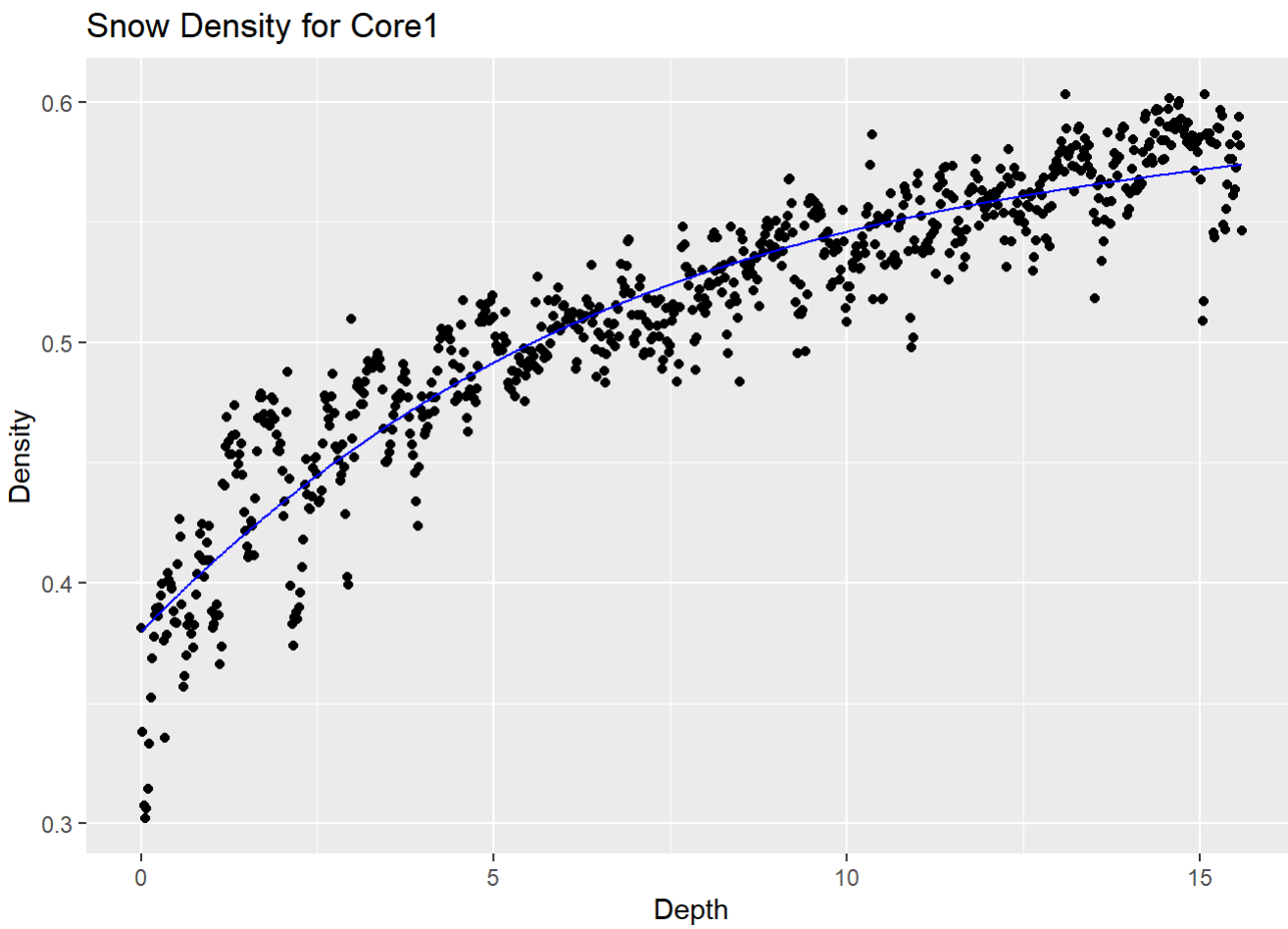
16. For this model, calculate the AIC.

```
AIC(lm_spline_cubint)
```

```
## [1] -57058.26
```

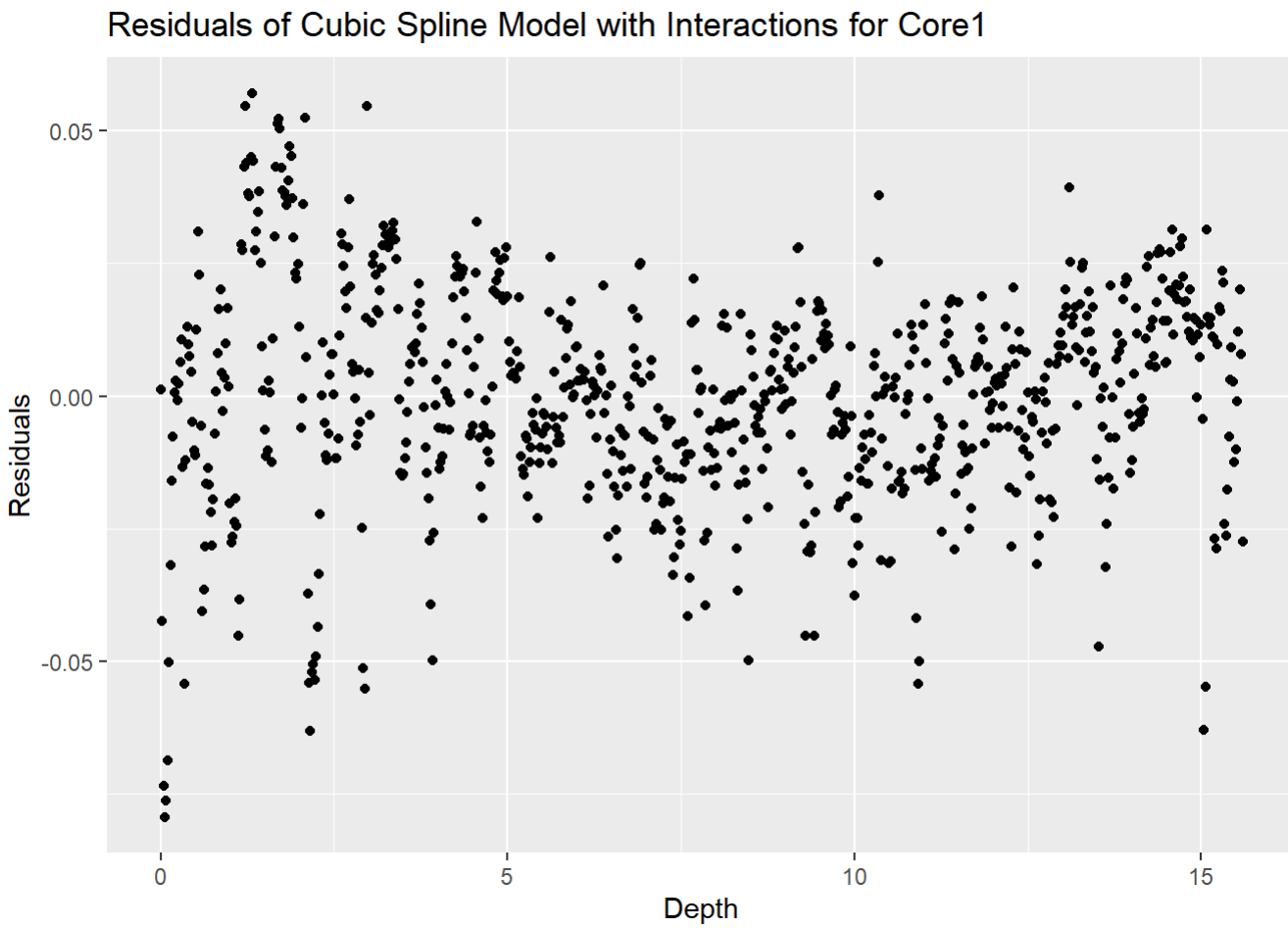
17. For the first site (Core = 1), plot the data and your model fit together to verify that it is effectively capturing the snow density curve. This should be the cubic spline model with interactions.

```
ggplot(subset(snow, Core == 1), aes(x = Depth, y = Density)) +
  geom_point() +
  geom_line(aes(y = pred.spline), color = "blue") +
  labs(title = "Snow Density for Core1",
       x = "Depth",
       y = "Density")
```



18. For the first site (Core = 1), plot the residuals of your model against depth. Comment on the pattern. This should be the cubic spline model with interactions.

```
ggplot(subset(snow, Core == 1), aes(x = Depth, y = resid.spline)) +
  geom_point() +
  labs(title = "Residuals of Cubic Spline Model with Interactions for Core1",
       x = "Depth",
       y = "Residuals")
```



The residuals here are all far closer around zero than the previous residuals graph. There is more negative deviation at the extremes, but still very good looking.

19. Of the six models fit, present your best model (in terms of AIC). Describe in words what your final model does. For example, is it one curve for all sites or a curve for each site? What are some advantages or disadvantages of your model?

The cubic polynomial has the lowest AIC and is thus the best model based on that metric. This model makes a cubic model for each core with no interaction. The coefficients cannot be interpreted which can be great disadvantages, but the model seems to fit the data well and therefore gives us the best predictive power.

20. Briefly summarize what you learned, personally, from this analysis about the statistics, model fitting process, etc.

I gained a better understanding of different non-linear modeling and how to test the fit of each model. These models include splines and polynomial functions.